ALGORITHME DE DIJKSTRA

F. M.

2022/2023

SOMMAIRE

1. REPRESENTATION D'UNE ARBORECENCE
2. BORDURE ET COCYCLE
3. PROPRIÉTÉ
4. L'ALGORITHME
5. EXTENSIONS

Table des pères et table des potentiels

Représentation d'une r-arborescence T incluse dans G par table des pères et table des potentiels :

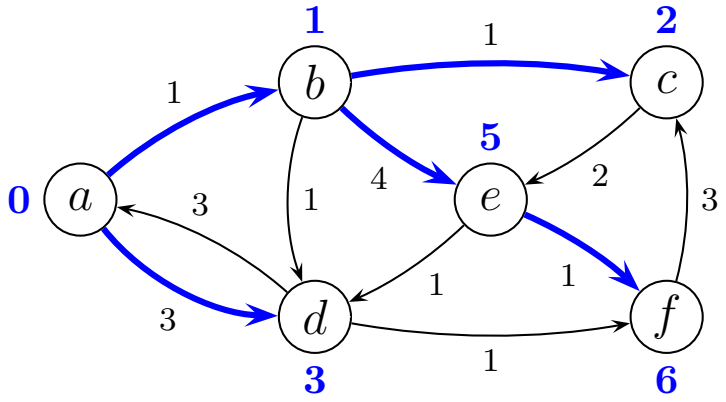
$$\forall s \in S,\ \mathcal{A}[s] = \begin{cases} \text{none} & \text{si } s = r \\ x & \text{si l'arc } (x, s) \in T \\ s & \text{si } s \notin T \end{cases}$$

$$\forall s \in S,\ \Pi[s] = \begin{cases} \text{poids du chemin de } r \text{ à } s \text{ dans } T & \text{si } s \in T \\ \infty & \text{si } s \notin T \end{cases}$$

Le sommet $\mathcal{A}[s]$ est appelé père du sommet s, le nombre $\Pi[s]$ est appelé potentiel du sommet s.

Illustration

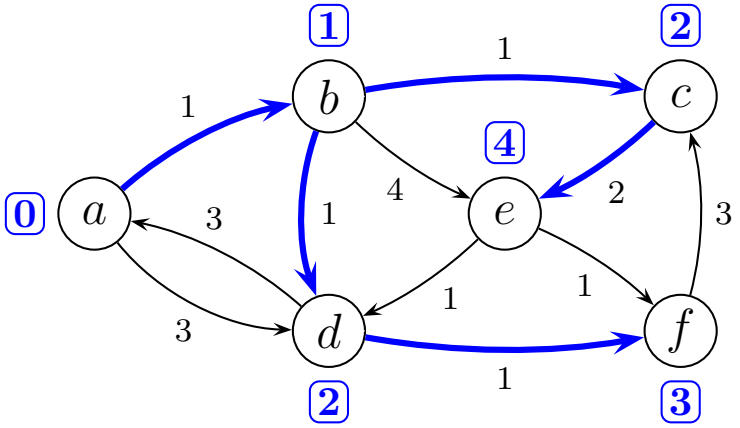
Une a-arborescence :



s	a	b	c	d	e	f
$\mathcal{A}[s]$	none	a	b	a	b	e
$\Pi[s]$	0	1	2	3	5	6

pour tout sommet s, $\Pi[s] \geq \textsc{Dist}(a, s)$

Une a-arborescence de chemins de poids minimum :



s	a	b	c	d	e	f
$\mathcal{A}[s]$	none	a	b	b	c	d
$\Pi[s]$	0	1	2	2	4	3

pour tout sommet s, $\Pi[s] = \textsc{Dist}(a, s)$

Définition 2.1

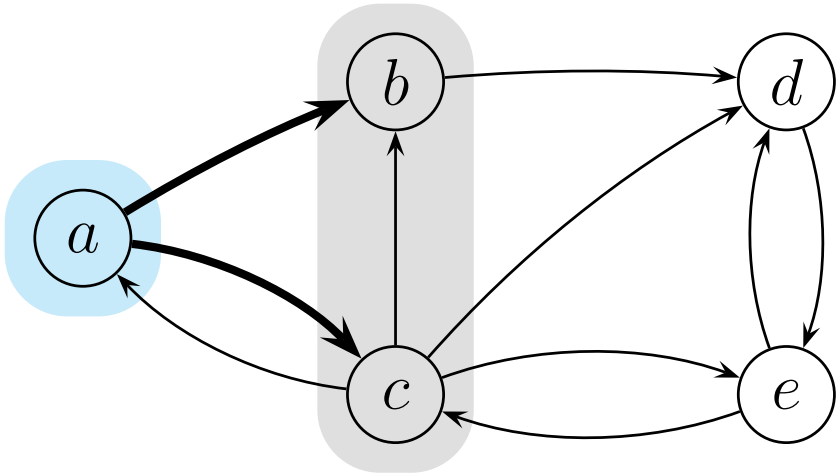
Soit $G = (S, A)$ un graphe orienté et soit E un sous-ensemble de S.

- L'ensemble noté $\boldsymbol{\mathcal{C}(E)}$ contenant les arcs dont l'origine se trouve dans E et le but à l'extérieur de E, est appelé **cocycle** de E.
  $\mathcal{C}(E)$ est l'ensemble des arcs *sortant* de E.

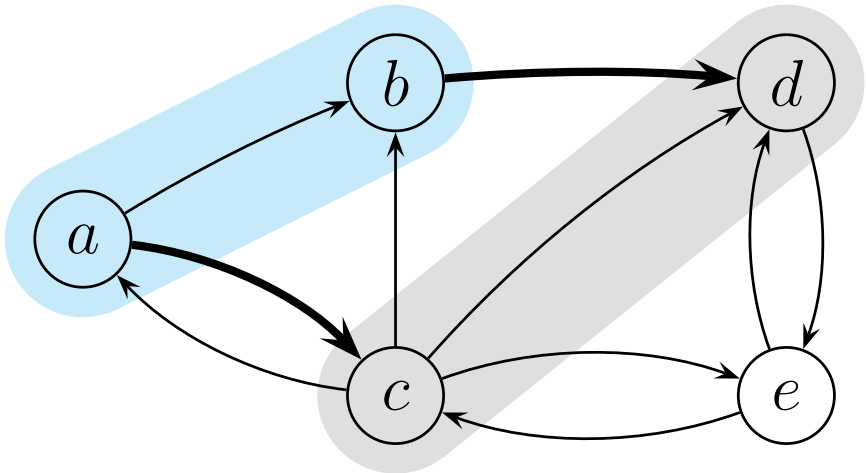
- L'ensemble noté $\boldsymbol{\mathcal{B}(E)}$ contenant les sommets n'appartenant pas à E mais possédant au moins un prédécesseur dans E, est appelé **bordure** de E.
  Les sommets de $\mathcal{B}(E)$ sont les buts des arcs de $\mathcal{C}(E)$. Les sommets de $\mathcal{B}(E)$ se trouvent en *bordure* de E (sur le diagramme).

Si $E = \{s\}$ on note simplement $\mathcal{C}(s)$ son cocycle et $\mathcal{B}(s)$ sa bordure.

Illustration



$$E = \{a\}$$
$$\mathcal{C}(a) = \{(a,b),(a,c)\}$$
$$\mathcal{B}(a) = \{b,c\} = V^+(a)$$



$$E = \{a,b\}$$
$$\mathcal{C}(E) = \{(a,c),(b,d)\}$$
$$\mathcal{B}(E) = \{c,d\}$$

3. PROPRIÉTÉ

Propriété 3.1

Soit $G = (S, A, P)$ est un graphe à **pondération positive** : $P(u) > 0$ pour tout arc u de A.

Soit T une r-arborescence partielle de chemins de poids minimum et soit E l'ensemble des sommets de T : $\forall x \in E$, $\Pi[x] = \textsc{Dist}(r, x)$.

Après relâchement des arcs de $\mathcal{C}(E)$, le sommet s de $\mathcal{B}(E)$ ayant le plus petit potentiel vérifie $\Pi[s] = \textsc{Dist}(r, s)$.

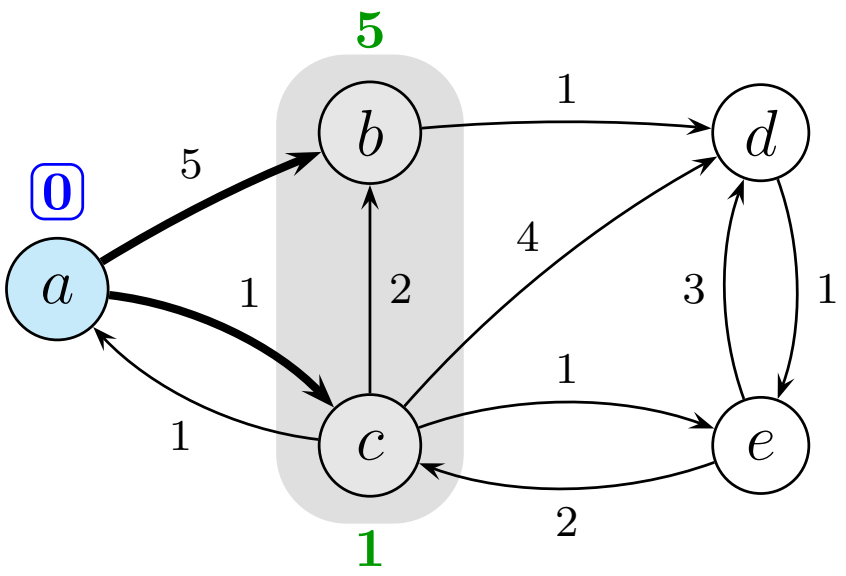
Soit s le sommet de $\mathcal{B}(E)$ ayant le plus petit potentiel après relâchement des arcs de $\mathcal{C}(E)$. Soit (x, s) le dernier arc d'un chemin élémentaire allant de r à s. Deux situations sont possibles :

- $x \in E$: (x, s) a été relâché et $\textsc{Dist}(r, x) + P(x, s) \geq \Pi[s]$ par hypothèse ;
- $x \notin E$: $\textsc{Dist}(r, x) \geq \Pi[s]$ puisque la pondération est positive.

$\Rightarrow$ $\Pi[s] = \textsc{Dist}(r, s)$

Illustration

$E = \{a\}$, $r = a$, T ne contient que a



$\mathcal{B}(a) = \{b, c\}$

$\mathcal{C}(a) = \{(a, b), (a, c)\}$

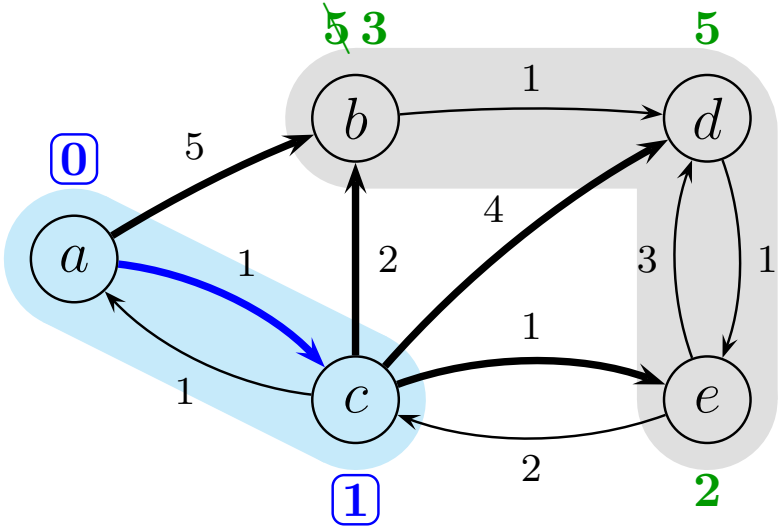
relâchement de (a, b) : $\Pi[b] \leftarrow 5$

relâchement de (a, c) : $\Pi[c] \leftarrow 1$

Propriété 3.1 : $\textsc{Dist}(a, c) = \Pi[c] = 1$ et $\mathcal{A}[c] \leftarrow a$

Illustration

$E = \{a, c\}$, $r = a$, T ne contient que l'arc (a, c)



$\mathcal{B}(E) = \{b, d, e\}$

$\mathcal{C}(E) = \{(a, b), (c, b), (c, d), (c, e)\}$

relâchement de (a, b) : $\Pi[b] \leftarrow 5$

relâchement de (c, b) : $\Pi[b] \leftarrow 3$

relâchement de (c, d) : $\Pi[d] \leftarrow 5$

relâchement de (c, e) : $\Pi[e] \leftarrow 2$

Prpopriété 3.1 : $\textsc{Dist}(a, e) = \Pi[e] = 2$ et $\mathcal{A}[e] = c$

Algorithme de DIJKSTRA

L'algorithme de Dijkstra détermine une r-arborescence de chemins de poids minimums dans un graphe $G = (S, A, P)$ à pondération positive. Cette arborescence est représentée par une table des pères $\mathcal{A}$ et par une table des potentiels Π.

L'algorithme utilise deux ensembles de sommets E et B. À chaque étape de l'algorithme :

- E contient les sommets appartenant à la r-arborescence de cpm en cours de construction.
- B correspond à la bordure de E.

L'algorithme repose sur la propriété 3.1. À chaque étape de l'algorithme, un sommet est extrait de B et ajouté à E (c.-à-d. à la r-arborescence) conformément à cette propriété.

Initialisations

✎ **initialisation des tables**

pour tout $s \in S$ **faire**

$\Pi[s] \leftarrow \infty$; $\mathcal{A}[s] \leftarrow s$

fin pour

$\Pi[r] \leftarrow 0$; $\mathcal{A}[r] \leftarrow \text{none}$

✎ **initialisation des ensembles**

$B \leftarrow \{r\}$; $E \leftarrow \varnothing$

En posant $B \leftarrow \{r\}$, on s'assure que le sommet r sera le premier sommet à basculer dans E.

Itérations

tant que $B \neq \varnothing$ **faire**

✎ **application de la propriété 3.1**

$s \leftarrow$ sommet de B ayant le plus petit potentiel

$B \leftarrow B \setminus \{s\}$; $E \leftarrow E \cup \{s\}$ ..... ✎ s passe de B à E

✎ **m.a.j. de B et relâchement des arcs d'origine s**

pour tout $x \in V^+(s) \cap \bar{E}$ **faire**

si $x \notin B$ **alors** $B \leftarrow B \cup \{x\}$

si $\Pi[s] + P(s,x) < \Pi[x]$ **alors**

$\Pi[x] \leftarrow \Pi[s] + P(s,x)$; $\mathcal{A}[x] \leftarrow s$

fin si

fin pour

fin tq

En réalité, l'ensemble E est inutile (on ne fait qu'ajouter des éléments à E). De plus, aucun relâchement ne peut améliorer le potentiel d'un sommet appartenant à E. L'ensemble E n'est utilisé ici que pour illustrer le fonctionnement de l'algorithme.

Itérations sans l'ensemble E

tant que $B \neq \varnothing$ **faire**

- ✎ **application de la propriété 3.1**
- $s \leftarrow$ sommet de B ayant le plus petit potentiel
- $B \leftarrow B \setminus \{s\}$ . . . . . ✎ s est extrait de B : $\Pi[s] = \textsc{Dist}(r, s)$
- ✎ **m.a.j. de B et relâchement des arcs d'origine s**
- **pour tout** $x \in V^+(s)$ **faire**
  - **si** $x \notin B$ **alors** $B \leftarrow B \cup \{x\}$
  - **si** $\Pi[s] + P(s, x) < \Pi[x]$ **alors**
    - $\Pi[x] \leftarrow \Pi[s] + P(s, x)$; $\mathcal{A}[x] \leftarrow s$
  - **fin si**
- **fin pour**

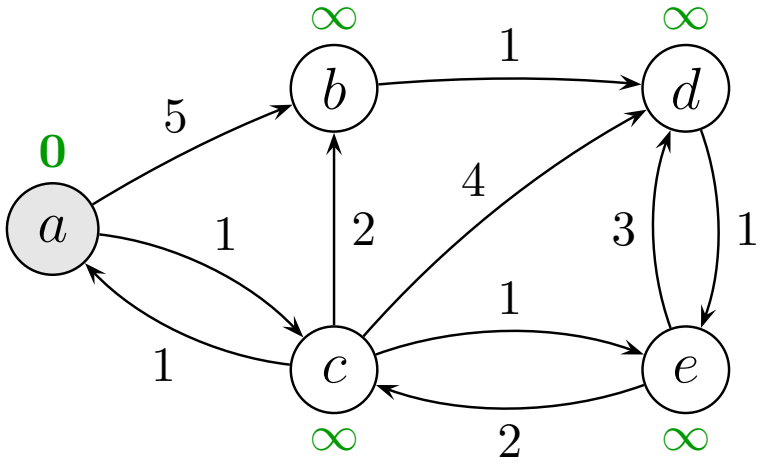
fin tq

Seul B (la bordure de E) est utilisé. À chaque itération, il suffit de relâcher tous les arcs d'origine s (les autres arcs du cocycle de E ont déjà été relâchés).

Trace de l'algorithme de Dijkstra

itér [0]

initialisation de Π, $\mathcal{A}$, B, E



E	a	b	c	d	e	B
$\varnothing$	0, none	∞, b	∞, c	∞, d	∞, e	a

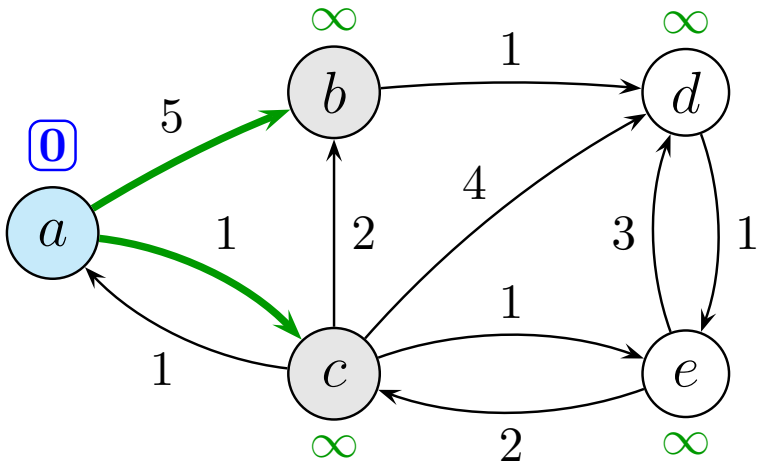
→ quand un sommet entre dans B, on le colore en gris sur le diagramme du graphe et on l'ajoute en colonne B du tableau

→ quand un sommet bascule dans E, on le colore en bleu sur le diagramme et on l'inscrit en colonne E du tableau

Trace de l'algorithme de Dijkstra

itér [1]

a bascule dans E
mise à jour de B
relâchements



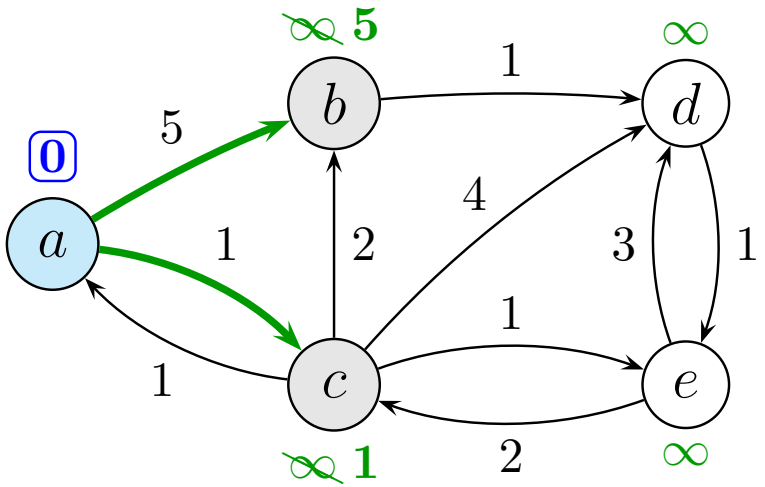
E	a	b	c	d	e	B
$\varnothing$	0, none	∞, b	∞, c	∞, d	∞, e	a
a	0, none					b c

→ à chaque itération, on annote le diagramme en fonction des relâchements et on reporte les résultats sur une nouvelle ligne du tableau

Trace de l'algorithme de Dijkstra

itér [1]

a bascule dans E
mise à jour de B
relâchements



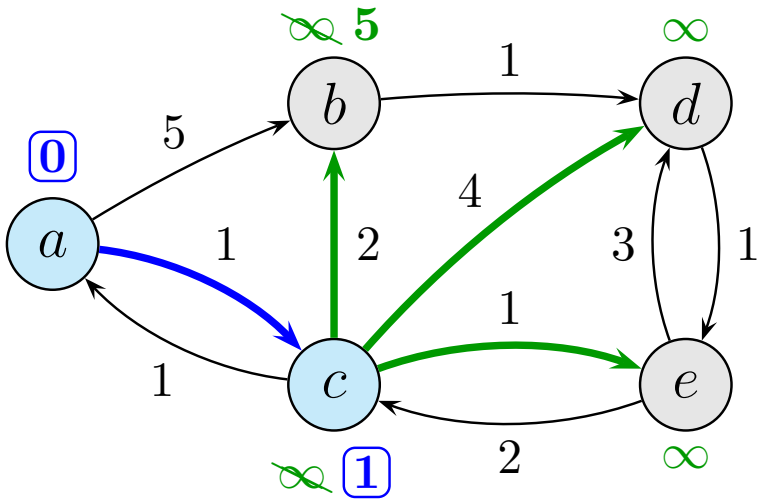
E	a	b	c	d	e	B
$\varnothing$	0, none	∞, b	∞, c	∞, d	∞, e	a
a	0, none	$\mathbf{5}, \boldsymbol{a}$	$\mathbf{1}, \boldsymbol{a}$	∞, d	∞, e	$b\ c$

→ à chaque itération, on annote le diagramme en fonction des relâchements et on reporte les résultats sur une nouvelle ligne du tableau

Trace de l'algorithme de Dijkstra

itér [2]

c bascule dans E
mise à jour de B
relâchements



E	a	b	c	d	e	B
$\varnothing$	0, none	∞, b	∞, c	∞, d	∞, e	a
a	0, none	$5, a$	$1, a$	∞, d	∞, e	$b\ c$
c			$1, a$			$b\ d\ e$

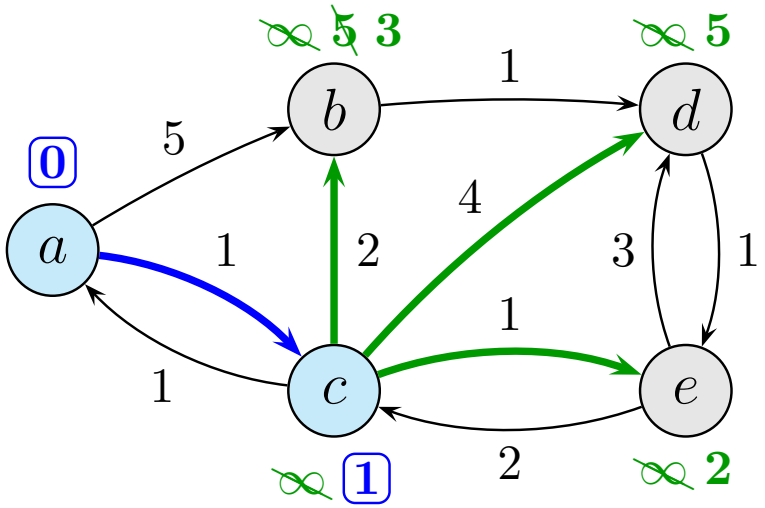
→ les arcs de l'arborescence des cpm sont colorés en bleu sur le diagrame

→ notons qu'il est inutile de relâcher (c, a) puisque a est dans E.

Trace de l'algorithme de Dijkstra

itér [2]

c bascule dans E
mise à jour de B
relâchements

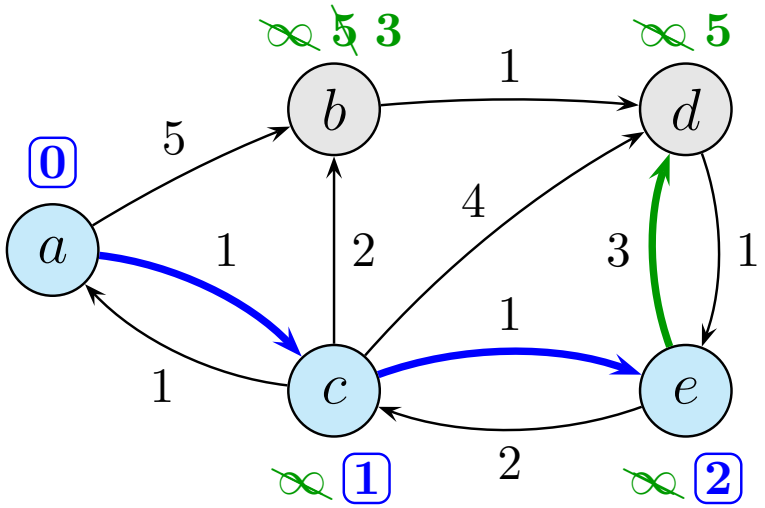


E	a	b	c	d	e	B
$\varnothing$	0, none	∞, b	∞, c	∞, d	∞, e	a
a	0, none	$5, a$	$1, a$	∞, d	∞, e	$b\ c$
c		$\mathbf{3, c}$	$1, a$	$\mathbf{5, c}$	$\mathbf{2, c}$	$b\ d\ e$

Trace de l'algorithme de Dijkstra

itér [3]

e bascule dans E
mise à jour de B
relâchements

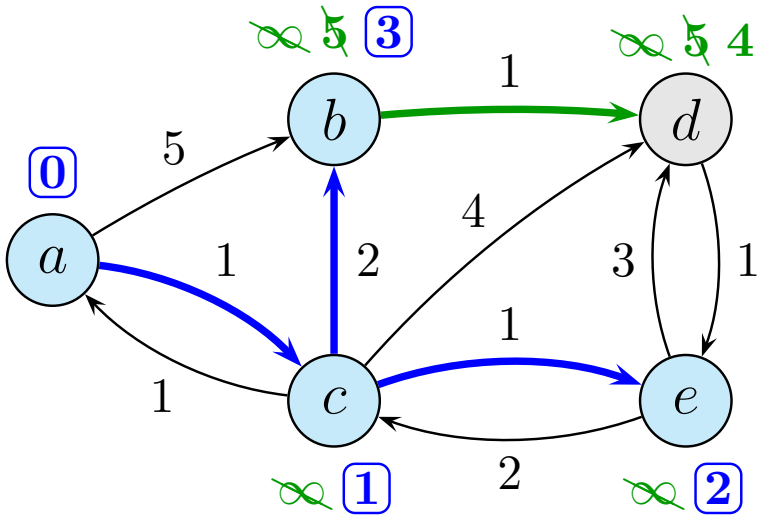


E	a	b	c	d	e	B
$\varnothing$	0, none	∞, b	∞, c	∞, d	∞, e	a
a	0, none	$5, a$	$1, a$	∞, d	∞, e	$b\ c$
c		$3, c$	$1, a$	$5, c$	$2, c$	$b\ d\ e$
e		$3, c$		$5, c$	$2, c$	$b\ d$

Trace de l'algorithme de Dijkstra

itér [4]

b bascule dans E
mise à jour de B
relâchements

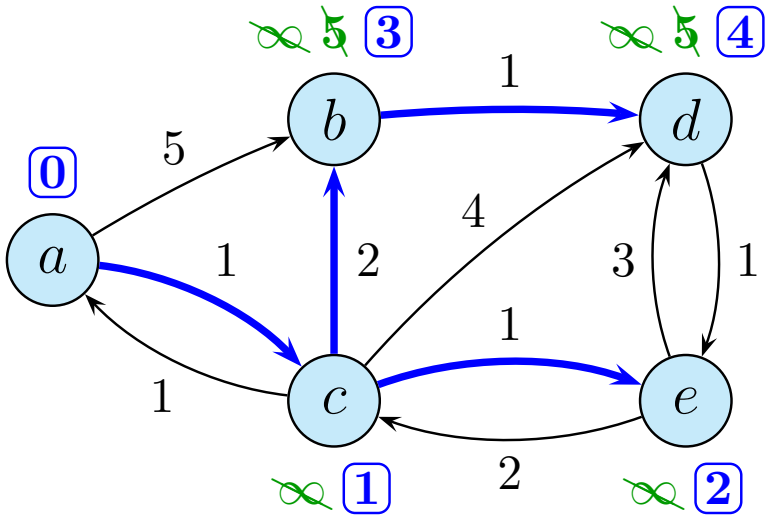


E	a	b	c	d	e	B
$\varnothing$	0, none	∞, b	∞, c	∞, d	∞, e	a
a	0, none	$5, a$	$1, a$	∞, d	∞, e	$b\ c$
c		$3, c$	$1, a$	$5, c$	$2, c$	$b\ d\ e$
e		$3, c$		$5, c$	$2, c$	$b\ d$
b		$3, c$		$\mathbf{4, b}$		d

Trace de l'algorithme de Dijkstra

itér [5]

d bascule dans E
B est vide
aucun relâchement

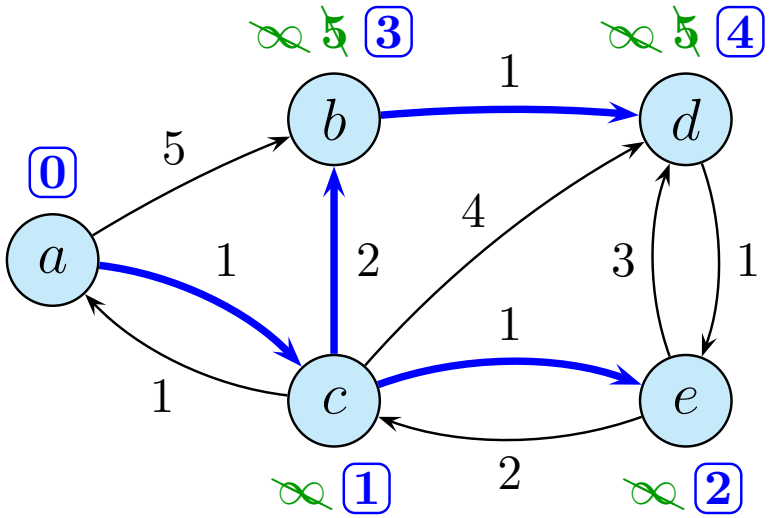


E	a	b	c	d	e	B
$\varnothing$	0, none	∞, b	∞, c	∞, d	∞, e	a
a	0, none	$5, a$	$1, a$	∞, d	∞, e	$b\ c$
c		$3, c$	$1, a$	$5, c$	$2, c$	$b\ d\ e$
e		$3, c$		$5, c$	$2, c$	$b\ d$
b		$3, c$		$4, b$		d
d				$4, b$		$\varnothing$

Trace de l'algorithme de Dijkstra

résultat

on a obtenu une
a-arborescence de cpm



E	a	b	c	d	e	B
$\varnothing$	0, none	∞, b	∞, c	∞, d	∞, e	a
a	0, none	$5, a$	$1, a$	∞, d	∞, e	$b\ c$
c		$3, c$	$1, a$	$5, c$	$2, c$	$b\ d\ e$
e		$3, c$		$5, c$	$2, c$	$b\ d$
b		$3, c$		$4, b$		d
d				$4, b$		$\varnothing$
$-$	0, none	$3, c$	$1, a$	$4, b$	$2, c$	$-$

Représentation de la bordure B

À chaque itération, il faut rechercher le sommet de l'ensemble B ayant le plus petit potentiel. Cette recherche est la partie la plus *coûteuse* de l'algorithme. L'efficacité de cette recherche dépend de la représentation de B.

Une solution simple est de représenter B par un tableau de sommets en le maintenant trié dans l'ordre croissant des potentiels tout au long de l'algorithme.

Habituellement, on utilise une structure de données particulière appelée **file de priorité** pour représenter l'ensemble B. Elle permet de stocker des éléments de sorte que l'élément de plus forte priorité soit toujours positionné en tête de file. La priorité d'un sommet est ici définie par son potentiel : plus le potentiel d'un sommet est petit et plus sa priorité sera grande.

Dans les deux cas, on accède directement au sommet de B ayant le plus petit potentiel (aucune recherche). Cependant, maintenir le tableau (insertions, décalages, suppressions) tout au long de l'algorithme est plus coûteux que de maintenir la file de priorité.

5. EXTENSIONS

- L'algorithme de Dijkstra peut être utilisé pour rechercher des cpm dans des graphes sans circuits mais, dans ce cas, on lui préfèrera l'algorithme de Bellman.
- L'algorithme de Dijkstra ne permet pas de rechercher des chemins de poids maximum : la pondération étant positive, tout circuit est absorbant.
- L'algorithme de Dijkstra peut être utilisé pour rechercher des chemins de probabilité maximum. Rappelons que dans cette situation, le poids d'un chemin est le produit des poids arcs du chemin. Le poids d'un chemin est alors une probabilité et un circuit ne peut être absorbant (son poids est inférieur à 1).
- L'algorithme de Dijkstra peut être utilisé pour rechercher des chaînes de poids minimum dans des graphes non orientés à pondération positive.